

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: A.Rohit

Roll No.: A24126552067

Date: 30-08-2026

1. TITLE

Object-Oriented Programming and Data Encapsulation using JavaScript ES6 Classes

2. AIM

To implement object-oriented programming principles in JavaScript by creating ES6 classes, instantiating objects, and dynamically injecting structured data into the Document Object Model (DOM).

3. OBJECTIVE

The objectives of this experiment are:

- To define JavaScript ES6 classes with constructors.
- To instantiate objects encapsulating student profile attributes.
- To extract user input from HTML form elements using JS.
- To validate form inputs dynamically preventing empty submissions.
- To construct HTML template strings and inject them into the DOM using innerHTML.

4. THEORY

JavaScript ES6 introduced the `class` keyword, providing syntactic sugar over the existing prototype-based inheritance model. Classes provide a much clearer and simpler syntax to create objects and deal with inheritance.

A constructor is a special method for creating and initializing an object created with a class. Data encapsulation is achieved by grouping attributes (like name, roll number, CGPA) into a single object instance.

The Document Object Model (DOM) allows JavaScript to read values from input fields, execute conditional validation logic, and dynamically update the page content without requiring a server request.

Application Flow:

User Input -> Button Click Event -> Form Validation -> Class Instantiation -> DOM Injection

5. ALGORITHM / PROCEDURE

- 1. Create a simple HTML UI containing input fields for Name, Roll No, Department, and CGPA.
- 2. Add a "Show Profile" button that triggers a JavaScript function on click.
- 3. Define an ES6 class named Student with a constructor initializing 4 properties.
- 4. Inside the click handler, retrieve values from the input fields using `document.getElementById().value`.
- 5. Implement an IF condition to check for empty strings and trigger an alert if validation fails.
- 6. Instantiate a new Student object passing the retrieved values as arguments.
- 7. Construct a template literal string mapping the object properties into HTML markup.
- 8. Inject the resulting string into a target `<div>` using the `innerHTML` property.
- 9. Test edge cases such as empty submissions and extreme values.

6. PROGRAM / SOURCE CODE

Source File: task3.html

File not found

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>class Student { constructor(...) }</code>	Defines an ES6 class blueprint with an initialization function.
<code>let student = new Student(...)</code>	Instantiates a new object from the Student class with specific data.
<code>document.getElementById('id').value</code>	Retrieves the current string value from an HTML input element.
<code>if (name === '') { alert(...); return; }</code>	Validates input and halts function execution if criteria are not met.
<code>display.innerHTML = `... \${var} ...`</code>	Updates the DOM content of an element dynamically using template literals.

8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Valid Object Instantiation	Provide all student details and click Show Profile	Profile card rendered dynamically with correct data	Pass
TC-02	Data Update & Re-render	Change CGPA to 9.00 and submit again	DOM successfully replaces old card with updated 9.00 card	Pass
TC-03	Empty Form Submission	Clear all inputs and click Show Profile	Validation halts execution and fires browser alert	Fail

9. OUTPUT

10. TEST CASES SCREENSHOTS

11. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.